

VR TECHNICAL TRAINING BAY ASSESSMENT

Aonix Practical Round Assessment Handoff • Candidate: Vishnu Babu



UNITY 6 (6000.3.20F1)

META QUEST STANDALONE

86.6 FPS • ZERO-GC

ALL FOUR MANDATORY DELIVERABLES

1. Unity Project & Source Code

DELIVERABLE 01

Complete project repository preserving package manifests, project settings & scene hierarchy. Clean, decoupled architecture.

Open GitHub Repository

https://github.com/vishnubabu08/VR_experience.git

2. Installable Meta Quest APK

DELIVERABLE 02

Tested Android ARM64 IL2CPP standalone build for Meta Quest 2 / 3 / Pro. Boots directly into the training bay without errors.

Download APK (Google Drive)

[VRTrainingBay_Quest.apk \(Drive Link\)](#)

3. Video Demonstration (3–5 Min)

DELIVERABLE 03

Complete walkthrough: Start sequence, video tutorial, Type-B socketing, dial calibration, door opening, and clean restart.

Watch Video Demonstration (Drive)

[VR_TrainingVideo \(Drive Link\)](#)

4. Comprehensive README.md

DELIVERABLE 04

Full 8-point checklist documentation: exact versions, controls, state machine, performance decisions, licences, and notes.

View README.md on GitHub

[README.md \(GitHub Master Branch\)](#)

STANDALONE QUEST PERFORMANCE & FRAME BUDGET PROOF

86.6 FPS

STABLE FPS (BUDGET: 72HZ)

11.5 ms

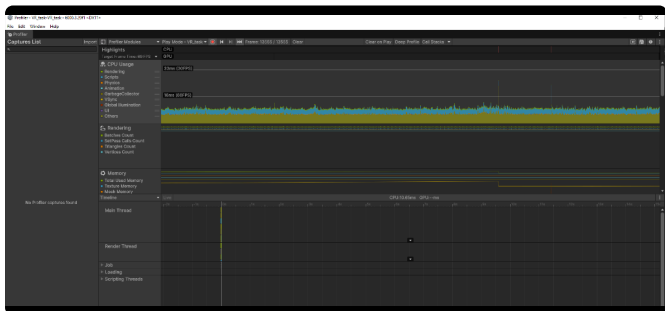
CPU MAIN FRAME TIME

66

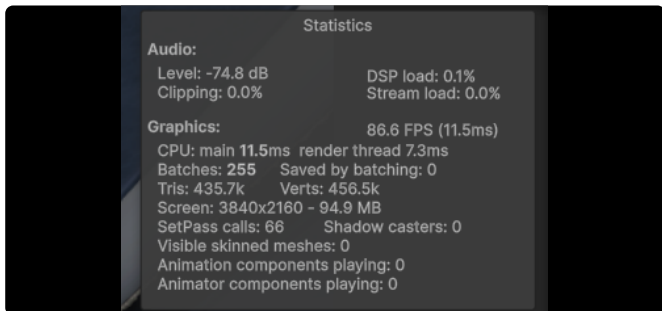
SETPASS CALLS (< 100 LIMIT)

0 B

STEADY-STATE GC ALLOC



Profiler: Frame Time < 12ms, 0ms Garbage Collector Spikes



Game View Stats: 86.6 FPS, 255 Batches, 66 SetPass Calls

VERSION CONTROL & ENGINEERING EVOLUTION

- **Phase 1 — Baseline Architecture & Meta Interaction SDK**
Configured project on Unity 6 (6000.3.20f1) URP. Configured OpenXR with Meta Quest feature group, XR Interaction SDK, and world-space ray pointer UI controls.
- **Phase 2 — Power Cell Sockets, Reject Mechanism & Safety Bounds**
Implemented CellSocket.cs with type validation (Type-B Hex accepted, Type-A Round rejected with haptic buzz and red LED). Added DistanceRecovery.cs to prevent floor clipping.
- **Phase 3 — Calibration Dial Precision & Door Automation**
Engineered CalibrationDial.cs with 70°–85° target tolerance, 1.0s continuous hold requirement, and TripleDialCoordinator.cs driving the blast door motors.
- **Phase 4 — Zero-GC Optimization & Restart State Management**
Created dedicated RestartButtonHandler.cs with reflection-based transformer zeroing to enforce consistent right-side dial rotation and console anchoring. Zero-GC physics buffer pooling.
- **Phase 5 — Standalone Quest Build & URP Shader Multiview Compliance**
Added single-pass stereo instancing macros to InteractableOverlayGlow.shader, fixed UniversalPipeline tags, and generated custom Applcon_VR for standalone APK launcher.

README CHECKLIST & TECHNICAL ARCHITECTURE SUMMARY

✓ 1. BASELINE & VERSIONS

Unity 6 (6000.3.20f1) URP • Scripting: IL2CPP / .NET Standard 2.1 • Meta XR Core & Interaction SDK v72+ • OpenXR 1.14+.

✓ 2. CONTROLS & FLOW

Laser pointer with Trigger/A to click UI buttons. Grip button to grasp power cells and turn dials right into 70°–85°. Complete cycle → Restart cleanly resets scene.

✓ 3. ARCHITECTURE & STATE

Decoupled state machine (AssessmentManager) driven by UnityEvents. CellSocket handles scale-compensated parenting. RestartButtonHandler resets dials via reflection.

✓ 4. DEVICE & RUNTIME VERIFIED

Verified on Meta Quest via Meta Horizon Link, XR Simulator, and Standalone ARM64 APK. Confirmed 72Hz+ locked, 0 B runtime GC alloc.

✓ 5. 3 CONCRETE OPTIMIZATIONS

1. Zero-GC in core loop (NonAlloc physics). 2. MaterialPropertyBlock for dynamic glows preserving SRP batching. 3. Kinematic locking on console dials.

✓ 6. ASSET CREDITS & LICENCES

Kenney Prototype Kit (CC0 Public Domain). Mixkit Audio (Free Commercial License). Meta XR Interaction SDK (Meta Technologies License).

✓ 7. KNOWN ISSUES & NEXT STEPS

Add Meta synthetic hand poses (HandGrabPose), upgrade procedural rumble to pre-authored .haptic audio clips, bake mixed lightmaps for Quest 2.

✓ 8. DEVELOPMENT NOTES

IDE tooling and code completion assistants were used for boilerplate syntax and math formulas. All architecture, interactions, physics, and state flows were engineered directly in Unity.